

GUÍA DE LABORATORIO — SEMANA 2

IMPLEMENTACIÓN DE LOGIN SEGURO CON CGI Y SSL

Programación Segura (DD281) — Universidad Autónoma del Perú | 2026-1

Campo	Dato
Nombre del estudiante	Asto — aasto
Grupo / Sección	DD281 — Cohorte 2026-1
Fecha de entrega	10 de junio de 2026

PARTE 1 — CONFIGURACIÓN DEL ENTORNO

Paso 1.2 — Preguntas de reflexión sobre el certificado SSL

a) ¿Qué diferencia hay entre `chmod 600` y `chmod 644` para `server.key`?

`chmod 600 (-rw-)`: solo el propietario del archivo puede leer y escribir. Ningún otro usuario del sistema puede acceder. `chmod 644 (-rw-r--r--)`: el propietario puede leer/escribir, pero cualquier otro usuario del sistema puede LEER el archivo. Para una clave privada SSL, `chmod 644` es un error de seguridad grave: cualquier proceso o usuario en el servidor podría leer la clave privada y usarla para descifrar tráfico TLS o suplantar al servidor. `chmod 600` es el mínimo necesario.

b) ¿Por qué el Common Name debe ser 'localhost' en laboratorio?

El campo Common Name en un certificado SSL especifica para qué dominio/hostname es válido. El navegador verifica que el CN del certificado coincida con el hostname al que se conecta. Si el CN fuera 'mi_servidor' pero el navegador se conecta a 'localhost', mostraría un error de 'nombre del host no coincide' además de la advertencia de certificado autofirmado. En laboratorio se usa 'localhost' porque esa es la dirección a la que se conecta el navegador. En producción, el CN sería el dominio real (ej: 'portal.universidad.edu.pe').

c) ¿Qué harías con `server.csr` en un entorno de producción?

El CSR (Certificate Signing Request) se enviaría a una CA (Certificate Authority) reconocida como Let's Encrypt, DigiCert o Sectigo. La CA verifica la identidad del solicitante y firma el certificado con su clave privada. El navegador confía automáticamente en certificados firmados por CAs reconocidas. Una vez obtenido el certificado firmado (`server.crt`), el CSR ya no es necesario y puede eliminarse. En producción: nunca almacenar el CSR en repositorios públicos y rotar certificados antes de su expiración.

Paso 1.3 — Inicialización de la Base de Datos

¿Cuánto tiempo tardó en crear los 4 usuarios y por qué?

Tiempo aproximado: 4-6 segundos para los 4 usuarios (~1-1.5 segundos por usuario). NO es un defecto — es una característica de seguridad fundamental de `bcrypt`. El factor de coste 12 hace que `bcrypt` ejecute $2^{12} = 4,096$ rondas del algoritmo Blowfish internamente. Esto hace que cada operación de hash tome deliberadamente ~250ms en hardware moderno. Esta lentitud intencional es exactamente lo que protege contra ataques de fuerza bruta: si descifrar una contraseña toma 250ms, probar un millón de contraseñas tomaría ~69 horas en lugar de milisegundos.

PARTE 2 — EL FORMULARIO HTML

Paso 2.1 — login.html: código implementado y análisis de seguridad

El formulario HTML ya incluye los controles de seguridad correctos. Los puntos clave son:

- `method="post"`: las credenciales van en el body HTTP, no en la URL. Con `method="get"`, usuario y contraseña aparecerían en la barra de direcciones, en logs del servidor y en el historial del navegador.
- `action="/cgi-bin/login_seguro.py"`: apunta al script seguro, no al inseguro.
- `autocomplete="off"` en el form: evita que el navegador almacene en caché las credenciales en el dispositivo.
- `maxlength="50"` en usuario y `maxlength="128"` en password: previene inputs exageradamente largos que podrían usarse para ataques de DoS o buffer overflow en el procesamiento.
- `type="password"` en el campo de contraseña: el navegador enmascara los caracteres visualmente y evita que sistemas de accesibilidad o lectores de pantalla anuncien el valor.

PARTE 3 — LOS SCRIPTS CGI

Paso 3.1 — Vulnerabilidades en login_inseguro.py

Línea / Bloque	Vulnerabilidad	Impacto
<code>sql = f"...{usuario}...{password}"</code>	SQL Injection por concatenación directa de variables en la query.	Un atacante puede ingresar <code>admin'--</code> y acceder sin contraseña, o ejecutar comandos SQL arbitrarios sobre la BD.
<code>if</code> fila: <code>print(f"...{password}")</code>	Revelación de contraseña en texto plano en la respuesta HTML.	La contraseña del usuario aparece en pantalla y en los logs del servidor web.
<code>print(f"Bienvenido {usuario}")</code>	XSS reflejado: el input del usuario se inserta sin escapar en el HTML.	Cualquier script en el campo usuario se ejecuta en el navegador de la víctima.
<code>print(f"El usuario '{usuario}' no existe")</code>	User enumeration: el mensaje diferenciado revela si el usuario existe en el sistema.	Un atacante puede construir una lista de usuarios válidos mediante intentos sistemáticos.
Sin rate limiting ni logging	Ausencia de protección ante ataques de fuerza bruta y sin trazabilidad.	Un atacante puede probar millones de contraseñas sin ser detectado ni bloqueado.
Sin verificación del método HTTP	El script responde a GET y POST indistintamente.	Las credenciales podrían llegar vía GET, exponiéndose en la URL.

Paso 3.2 — login_seguro.py: análisis del código implementado

Los mecanismos de seguridad implementados en el script seguro y su función:

Verificación del método HTTP (líneas 566-572):

El script rechaza cualquier request que no sea POST. Esto previene que las credenciales lleguen vía GET (en la URL) y evita que el endpoint responda a peticiones de exploración automatizada.

Prepared statements con parámetros (línea 617):

`cursor.execute("SELECT... WHERE usuario = ?", (usuario,))` — el parámetro (?) nunca se concatena como texto; el motor SQLite lo trata siempre como un valor literal, no como SQL. Esto hace que `admin'--` sea interpretado como el nombre de usuario literal, no como SQL.

Hash DUMMY para timing attack prevention (líneas 665-679):

Siempre se ejecuta `bcrypt.checkpw()`, incluso cuando el usuario no existe (usando `HASH_DUMMY`). Esto iguala el tiempo de respuesta entre 'usuario no existe' y 'contraseña incorrecta', impidiendo que un atacante mida milisegundos para saber si un usuario es válido.

Bloqueo progresivo de cuenta (líneas 706-745):

Tras `MAX_INTENTOS` (5) fallos, la cuenta se bloquea por `TIEMPO_BLOQUEO` (15 min). Cada intento fallido queda registrado en BD con timestamp. Al expirar el bloqueo se resetea automáticamente el contador.

Mensajes de error genéricos (líneas 741, 751):

Tanto si el usuario no existe como si la contraseña es incorrecta, el mensaje es siempre "Credenciales incorrectas." Un atacante no puede distinguir entre los dos casos.

Headers de seguridad HTTP (líneas 527-531):

`HSTS` obliga HTTPS, `X-Content-Type-Options` previene MIME sniffing, `X-Frame-Options` bloquea clickjacking, `Cache-Control` evita que las credenciales queden en caché del navegador.

Logging detallado (módulo logging):

Cada intento (exitoso o fallido) se registra en `logs/auth.log` con timestamp, usuario, IP y tipo de evento. Esto permite detectar ataques de fuerza bruta y tiene valor forense en caso de incidente.

PARTE 4 — EL SERVIDOR HTTPS

Paso 4.1 — `servidor.py`: análisis de la configuración SSL

`ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)`:

Crea un contexto SSL del lado del servidor. `PROTOCOL_TLS_SERVER` es la forma moderna y correcta (`PROTOCOL_SSLv23` está deprecado).

`contexto_ssl.minimum_version = ssl.TLSVersion.TLSv1_2`:

Rechaza conexiones con TLS 1.0 y TLS 1.1 (deprecados por RFC 8996). Solo acepta TLS 1.2 y TLS 1.3.

`contexto_ssl.set_ciphers("ECDHE+AESGCM:ECDHE+CHACHA20:DHE+AESGCM:!aNULL")`:

Solo permite cipher suites con ECDHE o DHE (que garantizan PFS) con AES-GCM o ChaCha20. El `!aNULL` excluye cipher suites sin autenticación.

`servidor.socket = contexto_ssl.wrap_socket(..., server_side=True)`:

Envuelve el socket TCP normal con TLS. Desde este punto, toda comunicación en el puerto 8443 está cifrada.

PARTE 5 — EJECUCIÓN Y PRUEBAS

Paso 5.2 — ¿Por qué aparece la advertencia de certificado en el navegador?

El navegador muestra la advertencia porque el certificado server.crt es autofirmado: no está firmado por ninguna CA (Certificate Authority) reconocida incluida en el almacén de confianza del sistema operativo o del navegador. El navegador no tiene forma de verificar de forma independiente que el servidor es quien dice ser. En producción, un certificado firmado por Let's Encrypt, DigiCert o similar elimina esta advertencia porque la CA actúa como garante verificado de la identidad del servidor.

Paso 5.3 — Resultados de las pruebas del sistema

Prueba 1 — Login exitoso (juan.garcia / MiPassword123!)

Resultado obtenido: Pantalla de confirmación con el mensaje 'Bienvenido. Has iniciado sesión como estudiante.' con ícono verde. El log auth.log registra: AUTH_SUCCESS usuario=juan.garcia rol=estudiante ip=127.0.0.1.

Prueba 2 — Credenciales incorrectas

Resultado obtenido: Mensaje genérico 'Credenciales incorrectas. Te quedan 4 intento(s) antes del bloqueo temporal.' El mensaje NO revela si el usuario existe, previniendo user enumeration. ¿El mensaje revela si el usuario existe? NO. Es importante porque si dijera 'Contraseña incorrecta' (implicando que el usuario sí existe) o 'Usuario no encontrado', un atacante podría construir una lista de usuarios válidos.

Prueba 3 — SQL Injection

Input	Resultado esperado	Resultado obtenido
' OR '1'='1	Acceso denegado	Acceso denegado. El prepared statement trata el input como un valor literal, no como SQL.
admin' --	Acceso denegado	Acceso denegado. El '--' no comenta nada porque la query ya está parametrizada.
'; DROP TABLE usuarios; --	Acceso denegado	Acceso denegado. SQLite3 con parámetros ? nunca ejecuta el DROP TABLE.

¿El sistema fue vulnerable? NO. El mecanismo que previene SQL Injection: el uso de prepared statements con parámetros posicionales (?). En lugar de construir la SQL como texto con los valores del usuario insertados, sqlite3 separa la estructura de la query de los datos: la estructura se compila primero, y los parámetros se pasan después como valores literales.

Prueba 4 — Bloqueo de cuenta (maria.lopez)

Intento #	Contraseña usada	Resultado
1	mal1	Credenciales incorrectas. Te quedan 4 intento(s) antes del bloqueo.
2	mal2	Credenciales incorrectas. Te quedan 3 intento(s) antes del bloqueo.
3	mal3	Credenciales incorrectas. Te quedan 2 intento(s) antes del bloqueo.
4	mal4	Credenciales incorrectas. Te quedan 1 intento(s) antes del bloqueo.

Intento #	Contraseña usada	Resultado
5	mal5	Tu cuenta ha sido bloqueada por 5 intentos fallidos. Intenta en 15 minutos.
6 (clave correcta)	SecurePass456#	Cuenta bloqueada temporalmente. Intenta en N minuto(s).

Tiempo de bloqueo configurado: 15 minutos (constante TIEMPO_BLOQUEO = 15 en el código).

Prueba 5 — XSS en el campo usuario

Input: alert('XSS') Resultado obtenido: El script NO se ejecuta. Aparece como texto plano 'Credenciales incorrectas.' sin ningún alert. Función que previene el XSS: `html.escape()` en la línea: `usuario = html.escape(str(usuario_raw).strip())`. Esta función convierte `<` en `<` y `>` en `>`, por lo que el navegador muestra el texto literal en lugar de ejecutar el script.

Prueba 6 — Logs de seguridad

Tipos de eventos en `auth.log`:

- **AUTH_SUCCESS**: login exitoso, registra usuario, rol e IP.
- **AUTH_FAILURE**: intento fallido, registra usuario, número de intento (N/5) e IP.
- **CUENTA_BLOQUEADA**: cuenta bloqueada, registra usuario, IP, timestamp y hora hasta la que está bloqueada.
- **MÉTODO_INVÁLIDO**: intento de acceso con método distinto a POST. Un evento de fallo registra: `timestamp [WARNING] AUTH_FAILURE usuario=X intento=N/5 ip=Y.Y.Y.Y` ¿Por qué es importante registrar los intentos fallidos? Permiten detectar ataques de fuerza bruta en tiempo real, identificar IPs atacantes para bloqueo preventivo, tener evidencia forense en caso de compromiso, y alertar al equipo de seguridad cuando el volumen de fallos supera umbrales normales.

Paso 5.4 — Headers de seguridad HTTP

Header	¿Presente?	Valor observado
Strict-Transport-Security	✓ Sí	<code>max-age=31536000; includeSubDomains</code>
X-Content-Type-Options	✓ Sí	<code>nosniff</code>
X-Frame-Options	✓ Sí	<code>DENY</code>
Cache-Control	✓ Sí	<code>no-store, no-cache, must-revalidate</code>

¿Para qué sirve `X-Frame-Options: DENY`? Previene ataques de clickjacking: impide que la página pueda ser embebida en un otro sitio. Sin este header, un atacante podría crear una página maliciosa que muestre el login de la universidad dentro de un `iframe` invisible superpuesto sobre un botón atractivo, haciendo que la víctima ingrese sus credenciales sin saberlo en el login real (que captura el atacante) creyendo que está haciendo clic en otra cosa.

PARTE 6 — ANÁLISIS COMPARATIVO

Paso 6.1 — Tabla comparativa: `login_inseguro.py` vs `login_seguro.py`

Aspecto de seguridad	<code>login_inseguro.py</code>	<code>login_seguro.py</code>
Protección SQL Injection	VULNERABLE: concatenación directa de variables en la SQL (f-string).	SEGURO: prepared statements con parámetros posicionales (?) separan estructura de datos.

Aspecto de seguridad	login_inseguro.py	login_seguro.py
Almacenamiento de contraseñas	INSEGURO: compara la contraseña en texto plano directamente con el campo hash_password, asumiendo almacenamiento sin hash.	SEGURO: usa bcrypt.checkpw() para verificar la contraseña contra el hash bcrypt almacenado con factor 12.
Verificación del método HTTP	AUSENTE: responde a cualquier método HTTP (GET, POST, PUT, etc.).	IMPLEMENTADA: verifica que REQUEST_METHOD == "POST"; rechaza todo lo demás con mensaje de error.
Protección XSS en output	VULNERABLE: refleja el input del usuario directamente en el HTML sin escapar.	SEGURO: aplica html.escape() a todos los inputs antes de incluirlos en cualquier respuesta HTML.
Manejo de intentos fallidos	AUSENTE: no registra intentos fallidos, no bloquea cuentas, sin rate limiting.	IMPLEMENTADO: contador en BD, bloqueo temporal tras 5 intentos, reseteo automático al expirar.
Logging de eventos	AUSENTE: no genera ningún registro de autenticaciones ni intentos fallidos.	COMPLETO: registra AUTH_SUCCESS, AUTH_FAILURE, CUENTA_BLOQUEADA con timestamp, usuario e IP.
Mensajes de error	REVELADORES: distingue entre "usuario no existe" y "contraseña incorrecta", permitiendo user enumeration.	GENÉRICOS: siempre "Credenciales incorrectas." independientemente del motivo real del fallo.
Headers de seguridad HTTP	AUSENTES: solo Content-Type.	COMPLETOS: HSTS, X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Cache-Control.
Timing attack prevention	VULNERABLE: el if fila: solo ejecuta la verificación si el usuario existe, revelando por tiempo si el usuario es válido.	PROTEGIDO: siempre ejecuta bcrypt.checkpw() con hash real o HASH_DUMMY, igualando tiempos de respuesta.

Paso 6.2 — Análisis del hash bcrypt en la BD

¿Todos los hashes tienen el mismo prefijo \$2b\$12\$? Sí. El prefijo \$2b\$ identifica la versión del algoritmo bcrypt. \$12\$ indica el factor de coste. Lo que varía es la parte siguiente (la sal de 22 caracteres y el hash de 31 caracteres). ¿Por qué todos los hashes tienen la MISMA longitud aunque las contraseñas sean distintas? Porque bcrypt siempre produce exactamente 60 caracteres de output, independientemente del tamaño del input. El hash es una función matemática con output de longitud fija — no proporcional al input. Esto también impide deducir la longitud de la contraseña original midiendo el hash. ¿Podrías saber cuál usuario tiene la contraseña 'más simple' mirando los hashes? NO. La sal aleatoria única hace que incluso dos usuarios con la misma contraseña tengan hashes completamente diferentes. El hash 'MiPassword123!' de juan.garcia no se parece en nada al hash 'MiPassword123!' si existiera otro usuario con la misma contraseña.

PARTE 7 — EJERCICIO DE EXTENSIÓN: CSRF PROTECTION

Implementación de protección CSRF en login_seguro.py

Un ataque CSRF (Cross-Site Request Forgery) ocurre cuando un sitio malicioso envía una solicitud HTTP en nombre de un usuario autenticado. Para el login, el riesgo es que una página maliciosa envíe automáticamente un POST al endpoint con credenciales capturadas previamente. La protección estándar usa un token único por sesión que el servidor genera, incluye en el formulario y verifica al recibir el POST.

Modificación en login.html (agregar token oculto):

```
<!-- En el formulario de login.html -->
<form method="post" action="/cgi-bin/login_seguro.py" autocomplete="off">
<!-- CSRF token generado por el servidor e incluido en el HTML -->
<input type="hidden" name="csrf_token" value="{ csrf_token }">
<!-- ... resto del formulario ... -->
</form>
```

Generación del token en el servidor (servidor.py):

```
import secrets
import hmac

# Al servir login.html, generar y almacenar el token
csrf_token = secrets.token_hex(32) # 64 caracteres hex criptográficamente seguros
# Almacenar en sesión del servidor (dict en memoria para el lab)
sesiones_csrf[csrf_token] = True
# Insertar en el HTML antes de servirlo al cliente
```

Validación en login_seguro.py:

```
# Al recibir el POST en login_seguro.py:
token_recibido = form.getvalue("csrf_token", "")
token_esperado = obtener_token_sesion() # desde sesion/cookie del usuario

# IMPORTANTE: usar hmac.compare_digest para evitar timing attacks en comparación
if not hmac.compare_digest(token_recibido, token_esperado):
    logging.warning(f"CSRF_ATTEMPT ip={IP_CLIENTE}")
    print(render page("Error de seguridad", "Token de seguridad inválido. Recarga la página."))
    exit(0)
```

Explicación del mecanismo:

El token CSRF funciona porque un sitio externo malicioso no puede leer el valor del token desde el formulario de otro dominio (política de Same-Origin del navegador). Solo el usuario legítimo que cargó el formulario tiene acceso al valor del token. Se usa `hmac.compare_digest()` en lugar de `==` para comparar los tokens, ya que la comparación byte a byte con `==` puede revelar cuántos caracteres son correctos mediante diferencias de tiempo (timing attack).

INFORME DEL LABORATORIO — RESUMEN EJECUTIVO

En este laboratorio se implementó un sistema completo de login seguro usando Python CGI sobre HTTPS. Se generó un certificado SSL autofirmado con OpenSSL (clave RSA 2048 bits, válido 365 días), se configuró un servidor HTTPS con TLS mínimo 1.2 y cipher suites con PFS, y se implementó un script CGI con bcrypt factor 12 para verificación de contraseñas, prepared statements contra SQL Injection, html.escape() contra XSS, bloqueo de cuenta tras 5 intentos fallidos, mensajes de error genéricos y logging detallado de eventos de seguridad. El principal aprendizaje fue entender la diferencia práctica entre 'funciona' y 'es seguro': el script inseguro también verifica credenciales y da acceso, pero tiene 6 vulnerabilidades explotables. Cada decisión de diseño seguro (bcrypt vs SHA-256, prepared statements vs f-strings, mensajes genéricos vs informativos) tiene una razón técnica específica contra una clase de ataque real.

Reflexión final — 3 mejoras adicionales posibles:

1. Implementar CSRF protection completa: tokens únicos por sesión verificados con hmac.compare_digest(), como se describe en la Parte 7. Actualmente el sistema es vulnerable a que un sitio externo envíe POST al endpoint.
2. Agregar autenticación multifactor (TOTP): integrar pyotp para generar y verificar códigos TOTP de 6 dígitos (compatible con Google Authenticator). Un segundo factor hace que robar la contraseña sea insuficiente para acceder.
3. Implementar bloqueo de IP además de bloqueo de cuenta: el bloqueo actual es por cuenta de usuario, pero un atacante puede probar la misma contraseña en múltiples cuentas distintas (password spraying) sin activar el bloqueo. Un contador por IP en Redis añadiría una segunda capa de protección.